agentic PBT × 真实 CVE:全流程与对应代码

把每个项目退回修复前一刻,「agent 有没有找到补丁修的那个缺陷」就有了标准答案

阶段一 · 建数据集

1 拉漏洞公告
筛出有 CVE、有修复提交的
`dataset/build_dataset.py`

↓ 2942 行 / 694 个包

2 定位被改的函数
在打补丁之前的源码上解析
`dataset/enrich_functions.py`

↓ `vuln_functions` 列

3 剔除平台假阴性
只在 Windows 成立的标掉
`dataset/platform_scan.py`

↓ 风险样本 17%

4 退回修复前一刻
切到修复提交的父提交
`dataset/materialize.py`

17 个样本

一份修复前的源码 + 一个被指向的模块
+ 一条已知的标准答案

阶段二 · 跑 agent

5 发起一次运行
串行,一次一个,撞配额就重试
`runs/serial.sh · go.sh`

↓ `claude -p "/hypo_v2 <模块>"`

6 agent 照着剧本走 8 步 `hypo_v2.md`

1. 看懂要测什么
2. 读实现
3. 写下代码承诺了什么 → claims.md ★
4. 只据 claims.md 想性质 ★
5. 写测试,每条标出对应哪条规约 ★
6. 跑测试,分诊失败
7. 写报告
8. 下结论

↓ 留下四样东西

`claims.md`	代码承诺了什么
`test_*.py`	性质测试
`bug_report_*.md`	缺陷报告与复现脚本
`claude_result.json`	回合数与花费

阶段三 · 量结果

7 还原每条性质
含通过的、和压根没想到的
`dataset/extract_properties.py`

↓ 每条性质一行

8 判定有没有找到
三个各自独立的信号
`dataset/classify_hit.py`

↓ 每次运行一行

9 5 次折成一档
稳定命中 / 不稳定 / 从未命中
`dataset/aggregate.py`

↓ 三档分类

10 事前可算的筛选量
不跑也能算,用来挑样本
`dataset/check_i3.py`

最终产出

`table1_cve.csv` `table2_results.csv`
`table3_stability.csv`

方框右下角 = 执行这一步的代码文件　　★ = 为「让规约可追溯」新增的步骤